Filas

0004617A/4710A – Estrutura de Dados I

Prof. Me. Giovani Candido

Prof. Me. Luiz Fernando Sementille

BCC/BSI - Bauru

Definição de Fila

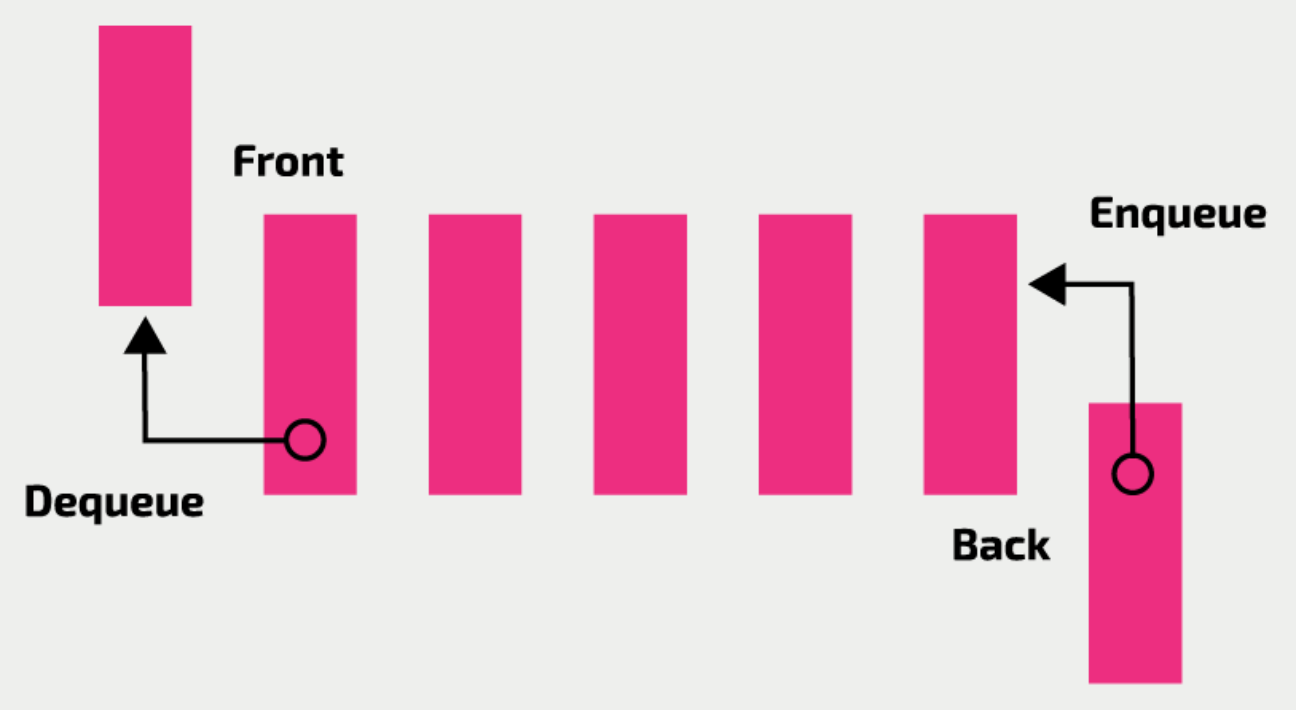
Filas (Queues)

- Assim como a **pilha**, a **fila** é um caso especial de lista, portanto, uma **estrutura linear**.
- Faz analogia a filas de espera, como a fila do caixa eletrônico ou a fila do açougue.
  - A ordem de chegada dos elementos é sempre preservada: quem chega primeiro ocupa a frente da fila (**front**), enquanto os novos elementos são alocados na parte de trás da fila (**back**).
  - Elementos que chegam depois **devem aguardar o avanço da fila** para serem atendidos.
- Dessa forma, a fila implementa uma política de **primeiro a entrar, primeiro a sair**.
  - No inglês, também conhecida por **FIFO** (**First-In, First-Out**).

Filas (Queues)

- O **TAD de uma fila** geralmente implementa operações como:
  - **Verificar se a fila está vazia**.
  - **Verificar se a fila está cheia**.
  - **Enfileirar (enqueue/push):** inserir um novo elemento no **final (back)** da fila.
  - **Desenfileirar (dequeue/pop):** remover um elemento do **início (front)** da fila.
  - **Espiar (peek/front):** consultar o elemento no **início da fila** sem removê-lo.
  - **Determinar o tamanho da fila**.
- Para fins de teste, também pode existir uma operação para exibir a fila toda.

Filas (Queues)



Filas (Queues)

- Em termos de implementação, a fila é uma lista na qual a inserção ocorre **somente em um extremo (final)**, enquanto a remoção ocorre **no extremo oposto (início/frente)**.
  - Diferentemente da pilha, a implementação da fila utiliza os dois extremos da lista.
- A fila pode ser implementada de forma **estática (vetor)** ou **dinâmica (lista encadeada)**.
  - Mesmo que a estrutura base (ex.: **vetor**) permita acesso direto, a fila deve sempre respeitar a política **FIFO**: **para acessar um elemento, todos os anteriores precisam ser removidos**.
- Dependendo da abordagem, a complexidade das operações da fila pode ser **O(1)**.

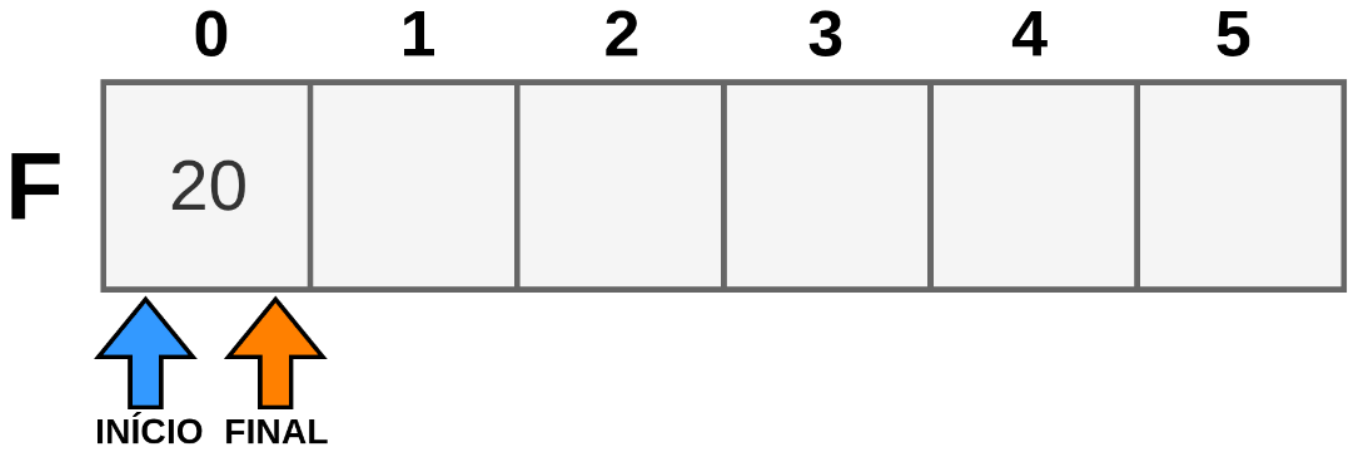
Filas Estáticas

Filas Estáticas

- Para implementar a fila, dois indicadores são necessários: **início/frente** e **fim**.
- À medida que elementos são inseridos ou removidos, esses indicadores se deslocam, o que pode gerar uma **falsa impressão de memória esgotada**.
  - Quando elementos são adicionados, o indicador de **fim avança**; quando são removidos, o indicador de **início avança**. Com o tempo, isso pode reduzir o espaço útil do vetor.
- Porém, existem dois modos de contornar esse problema:
  - Utilizar **deslocamentos** para recuperar o espaço útil do vetor.
  - Implementar um **vetor circular**.

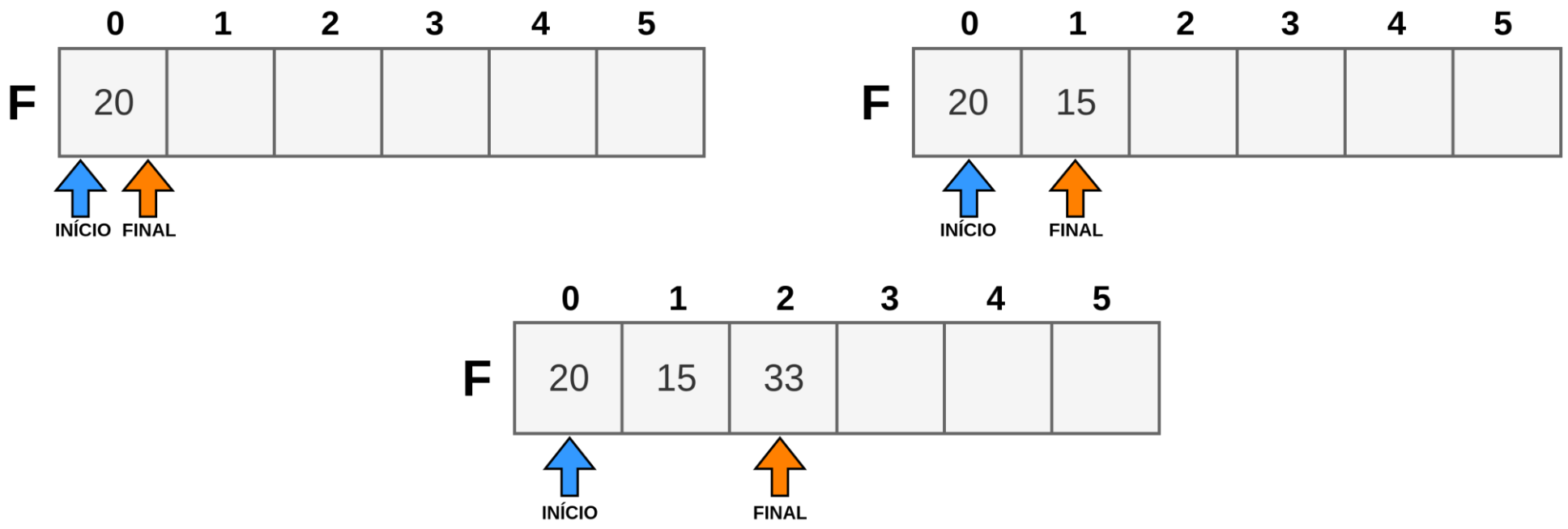
Filas Estáticas

- Inicialmente, os dois indicadores apontam para o **único elemento da fila**. À medida que novos elementos são **enfileirados**, o **indicador de fim avança**:



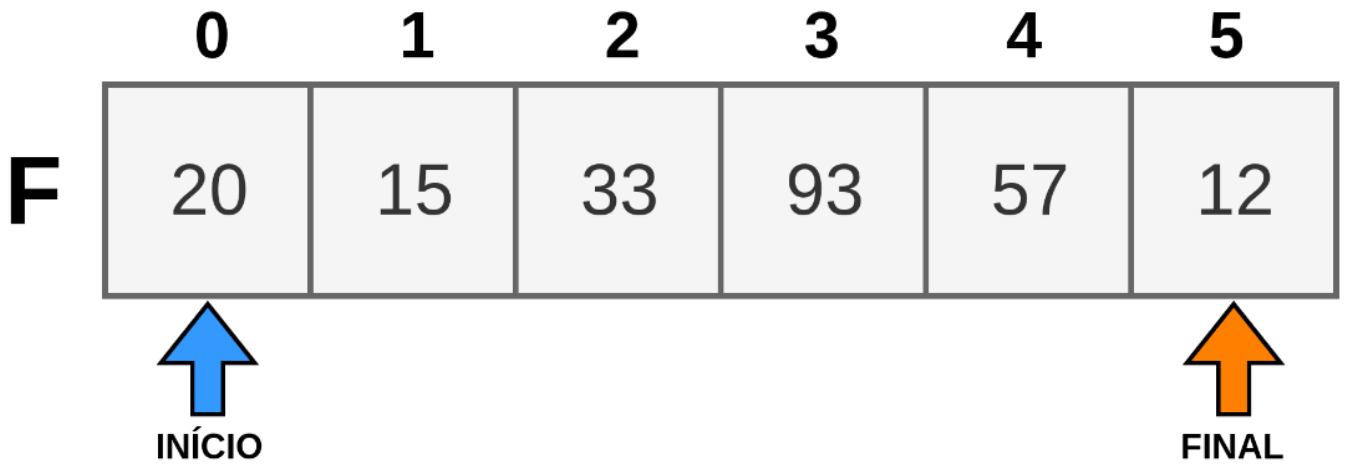
Filas Estáticas

- Inicialmente, os dois indicadores apontam para o **único elemento da fila**. À medida que novos elementos são **enfileirados**, o **indicador de fim avança**:



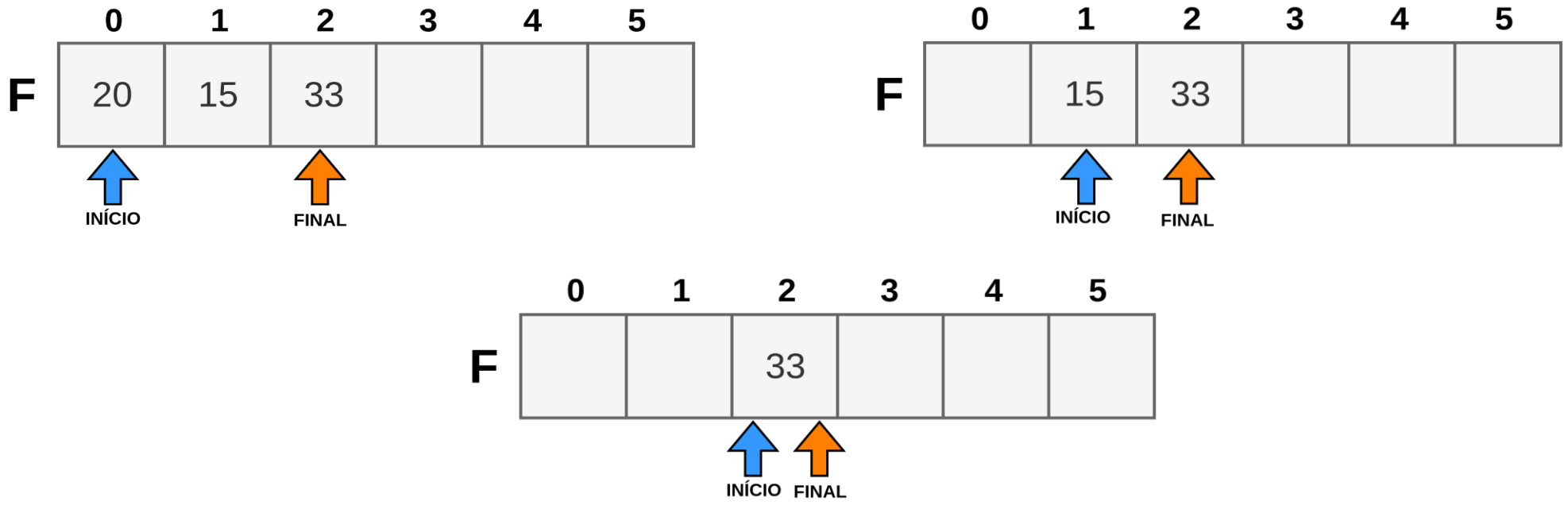
Filas Estáticas

- A imagem abaixo mostra a fila após várias inserções que preencheram o vetor; o **início** permanece na **primeira posição**, enquanto o **fim alcançou a última posição**.



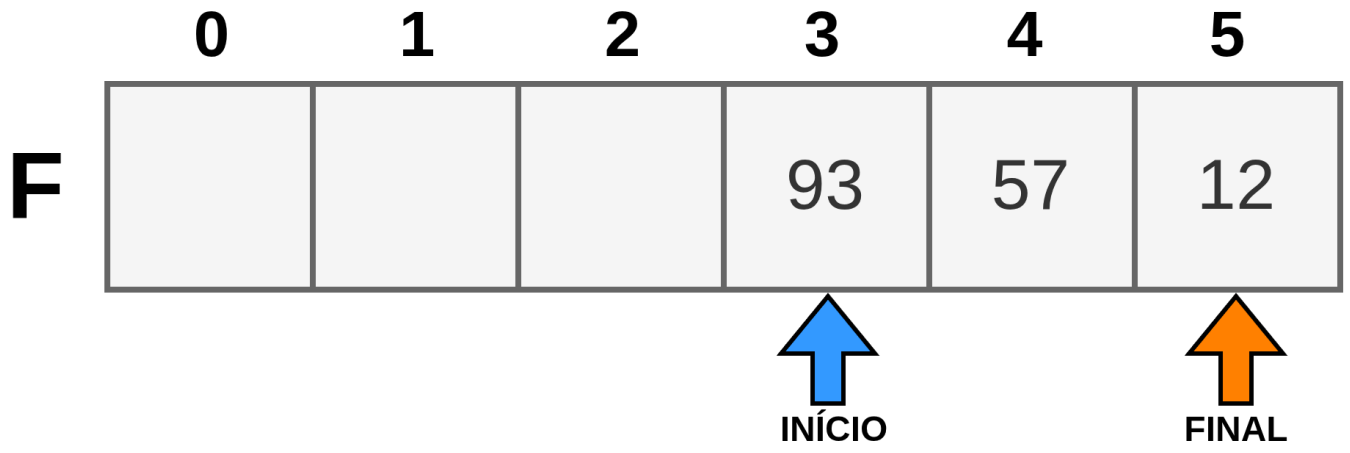
Filas Estáticas

- À medida que elementos são **removidos da fila**, o **indicador de início avança posições dentro do vetor**, enquanto o **indicador de fim permanece inalterado**.



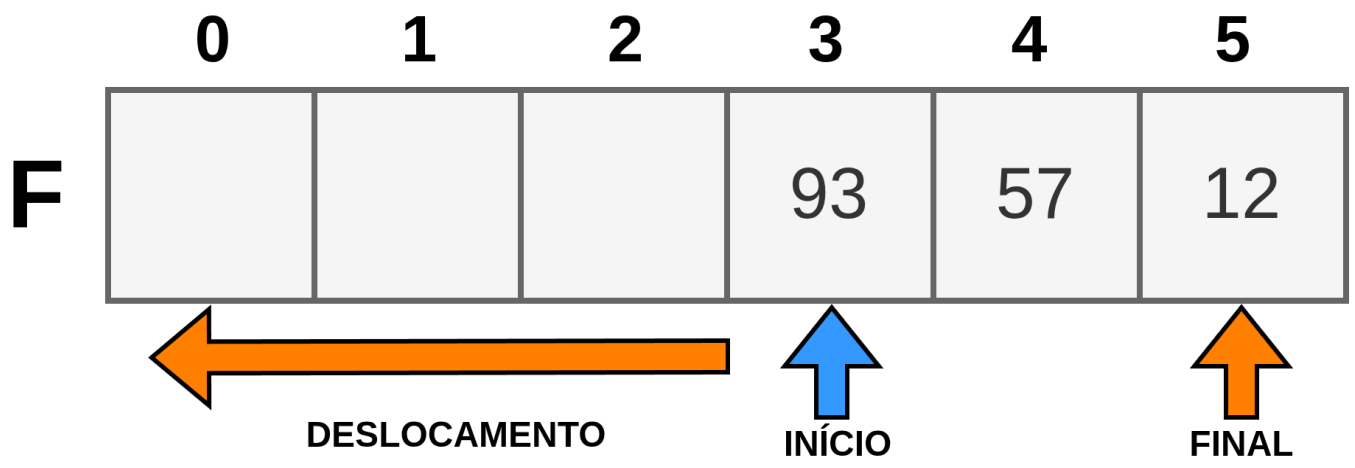
Filas Estáticas

- Após remover alguns elementos do início da fila, o vetor pode atingir a seguinte situação: **embora haja espaço livre no início, novas inserções não são possíveis**.



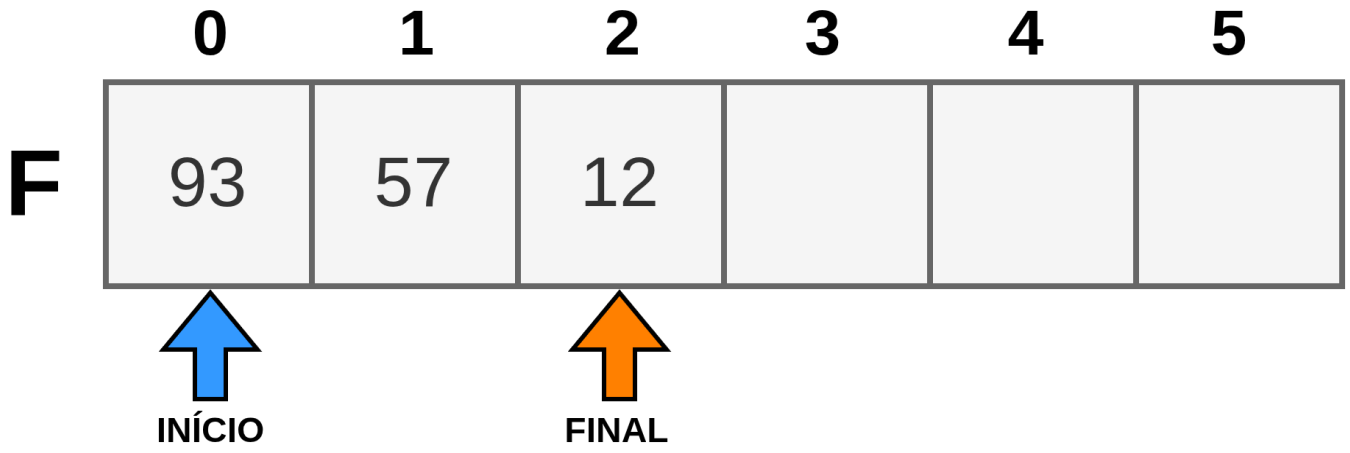
Filas Estáticas

- Para resolver esse problema, quando uma nova inserção for realizada e **não houver espaço disponível no final do vetor**, os elementos podem ser **deslocados para o início**.



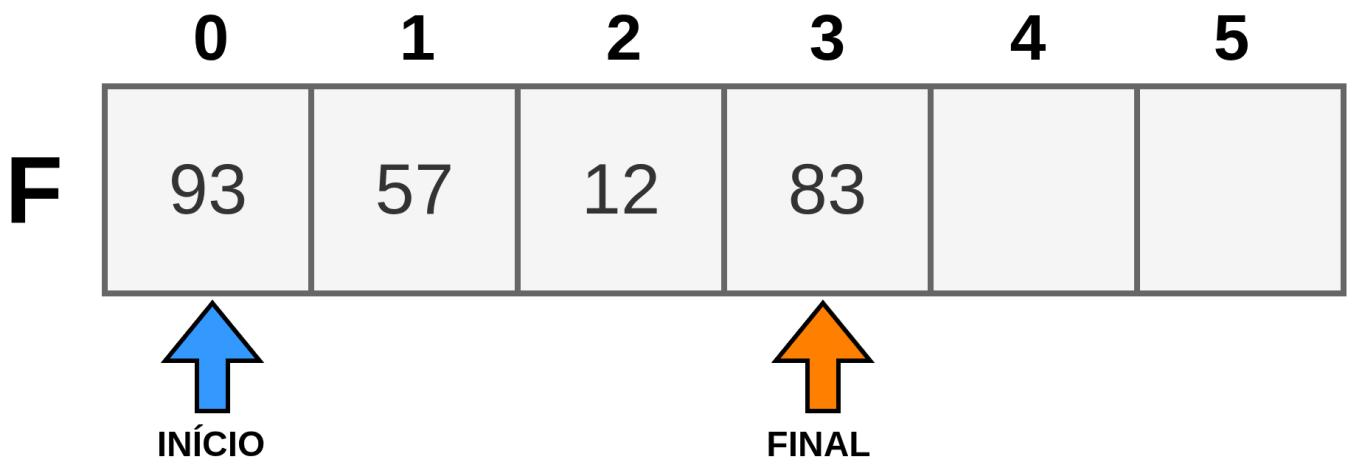
Filas Estáticas

- Após o deslocamento, o indicador de **início** volta a ocupar a **primeira posição do vetor**, permitindo novamente inserções no final da fila.



Filas Estáticas

- Após o deslocamento, o indicador de **início** volta a ocupar a **primeira posição do vetor**, permitindo novamente inserções no final da fila.



Filas Estáticas

- A imagem abaixo apresenta a definição de um tipo **Fila** utilizando uma estrutura **(struct)** para armazenar os itens e os indicadores de início e final:

```c
#define TAMANHO_FILA 10

typedef struct estrutura_fila {
    int itens[TAMANHO_FILA];
    int inicio, final;
} Fila;


void inicializar_fila(Fila *fila) {
    fila->inicio = 0;
    fila->final = -1;
}
```

Filas Estáticas

- A imagem abaixo apresenta a definição de um tipo **Fila** utilizando uma estrutura **(struct)** para armazenar os itens e os indicadores de início e final:

```c
#define QUEUE_SIZE 10

typedef struct queue {
    int items[QUEUE_SIZE];
    int front, back;
} Queue;


void initialize_queue(Queue *queue) {
    queue->front = 0;
    queue->back = -1;
}
```

Filas Estáticas

- Considerando essa definição, podemos implementar as seguintes operações:

```
int esta_vazia(Fila fila);
int esta_cheia(Fila fila);
int tamanho_fila(Fila fila);
int enfileirar(Fila *fila, int item);
int desenfileirar(Fila *fila, int *item);
int espiar(Fila fila, int *item_inicio);
void mostrar_fila(Fila fila);
```

Filas Estáticas

- Em inglês, pode-se adotar a tradução literal **enqueue** para enfileirar e **dequeue** para desenfileirar, como encontrado em alguns livros:

```
int is_empty(Queue queue);
int is_full(Queue queue);
int queue_size(Queue queue);
int enqueue(Queue *queue, int item);
int dequeue(Queue *queue, int *item);
int peek(Queue queue, int *front_item);
void show_queue(Queue queue);
```

Filas Estáticas

- Porém, é comum seguir a mesma convenção utilizada anteriormente com **push** para inserir e **pop** para remover, como encontrado em algumas bibliotecas:

```
int is_empty(Queue queue);
int is_full(Queue queue);
int queue_size(Queue queue);
int push(Queue *queue, int item);
int pop(Queue *queue, int *item);
int peek(Queue queue, int *front_item);
void show_queue(Queue queue);
```

Filas Estáticas

- Na implementação estática, a verificação de fila vazia consiste em **checar se o final da fila possui o valor –1**, isto é, não armazena nenhum índice válido do vetor.

```
int esta_vazia(Fila fila) {
    return fila.final == -1;
}
```

Filas Estáticas

- Para verificar se a fila está cheia, é necessário ver se o **início da fila está no índice 0** do vetor e se o **final da fila está no último índice válido do vetor**.

```
int esta_cheia(Fila fila) {
    return fila.inicio == 0 && fila.final == TAMANHO_FILA - 1;
}
```

Filas Estáticas

- Para obter o tamanho atual da fila, precisamos checar, primeiramente, se a fila está vazia.
  - Se estiver, então o **tamanho será 0**;
  - Senão, será dado por: **índice do final da fila - índice do início da fila + 1**.

```
int tamanho_fila(Fila fila) {
    return esta_vazia(fila) ? 0 : fila.final - fila.inicio + 1;
}
```

Filas Estáticas

- A função enfileirar nada mais é do que uma **inserção de elementos no final do vetor**.
- Porém, além de verificar se a fila está cheia, **precisamos verificar se o índice do final já está no último índice válido da fila.**
  - **Se estiver**, precisamos deslocar os elementos da fila para o início do vetor antes de inserir.
  - **Se não estiver**, apenas inserimos no final.

```c
int enfileirar(Fila *fila, int item) {
    if (esta_cheia(*fila)) {
        return 0;
    }

    if (fila->final == TAMANHO_FILA - 1) {
        for (int i = fila->inicio; i <= fila->final; i++) {
            fila->itens[i - fila->inicio] = fila->itens[i];
        }

        fila->final = fila->final - fila->inicio;
        fila->inicio = 0;
    }

    fila->itens[++fila->final] = item;

    return 1;
}
```

Filas Estáticas

- Já a função desenfileirar é mais simples, pois **apenas temos que verificar se a fila está vazia** e **então remover o elemento.**
- Porém, **se esse elemento é o único da fila**, a fila ficará vazia após sua remoção e, para reaproveitarmos os espaços do vetor, **inicializamos a fila novamente**.

```
int desenfileirar(Fila *fila, int *item) {
    if (esta_vazia(*fila)) {
        return 0;
    }

    *item = fila->itens[fila->inicio++];

    if (fila->inicio > fila->final) {
        inicializar_fila(fila);
    }

    return 1;
}
```

Filas Estáticas

- Assim como em pilha, a função espiar também é interessante.
- Neste caso, podemos visualizar o elemento que está no início da fila sem removê-lo.

```
int espiar(Fila fila, int *item_inicio) {
    if (esta_vazia(fila)) {
        return 0;
    }

    *item_inicio = fila.itens[fila.inicio];

    return 1;
}
```

Filas Estáticas

- Uma função menos usual, mas muito útil, é a de mostrar todos os itens da fila.
- De forma análoga ao que acontece em pilhas, essa implementação **fere o conceito de fila**, pois estamos acessando seus elementos de outra forma que não por meio da função desenfileirar.

```
void mostrar_fila(Fila fila) {
    if (esta_vazia(fila)) {
        printf("A fila está vazia\n");
        return;
    }

    printf("Itens da fila: ");

    for (int i = fila.inicio; i <= fila.final; i++) {
        printf("%d ", fila.itens[i]);
    }

    printf("\n");
}
```

Filas Estáticas

- A **forma mais correta** de mostrar a fila é desenfileirando cada elemento.
- Neste exemplo, como passamos a fila por valor à função mostrar_fila, podemos manipulá-la sem se preocupar com alterações na fila real.

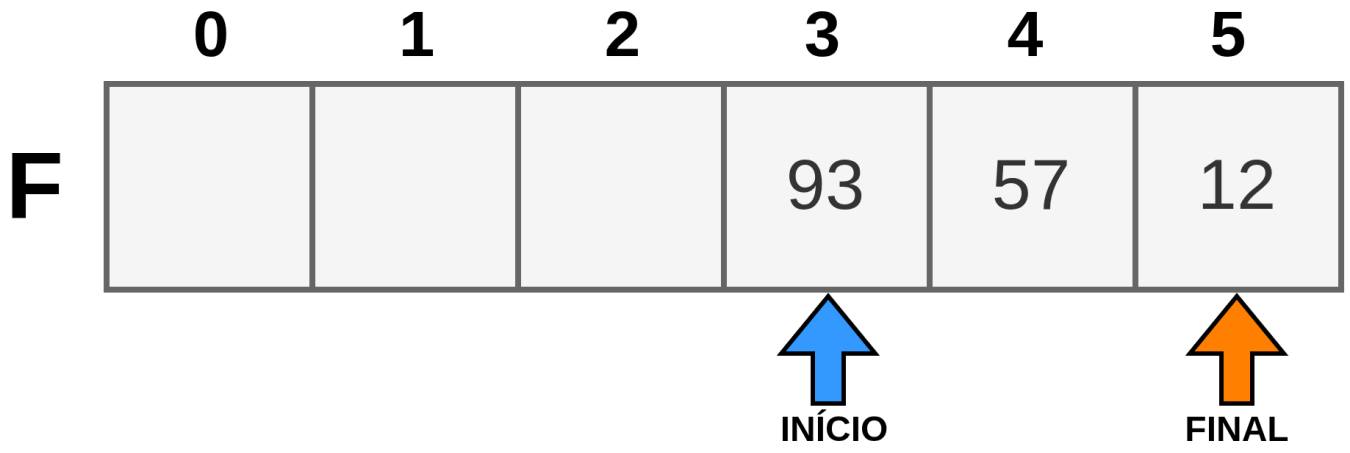
```c
void mostrar_fila(Fila fila) {
    if (esta_vazia(fila)) {
        printf("A fila está vazia\n");
        return;
    }

    printf("Itens da fila: ");

    int item;

    while (!esta_vazia(fila)) {
        desenfileirar(&fila, &item);
        printf("%d ", item);
    }

    printf("\n");
}
```

Filas Estáticas

- Como vimos na abordagem anterior, a inserção pode ocorrer em **O(n)** quando é necessário deslocar os elementos para o início do vetor.
- Para evitar esse problema e manter a complexidade **O(1)**, a fila estática é frequentemente implementada como um vetor circular, embora seja uma abordagem menos intuitiva.
- Em um **vetor circular**, o indicador de **final da fila** pode migrar para o **início do vetor** quando houver **posições disponíveis**, permitindo que novas inserções sejam realizadas ali.

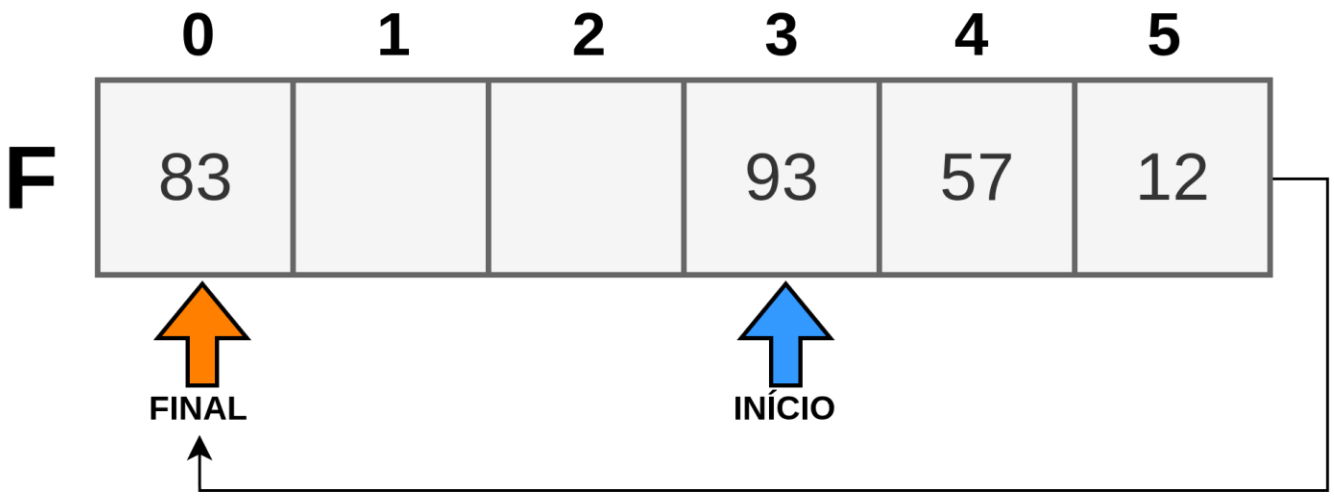
Filas Estáticas

- Considere a seguinte situação apresentada anteriormente, em que o **espaço livre no vetor parece limitado**, pois o indicador de início foi movido algumas posições para frente.



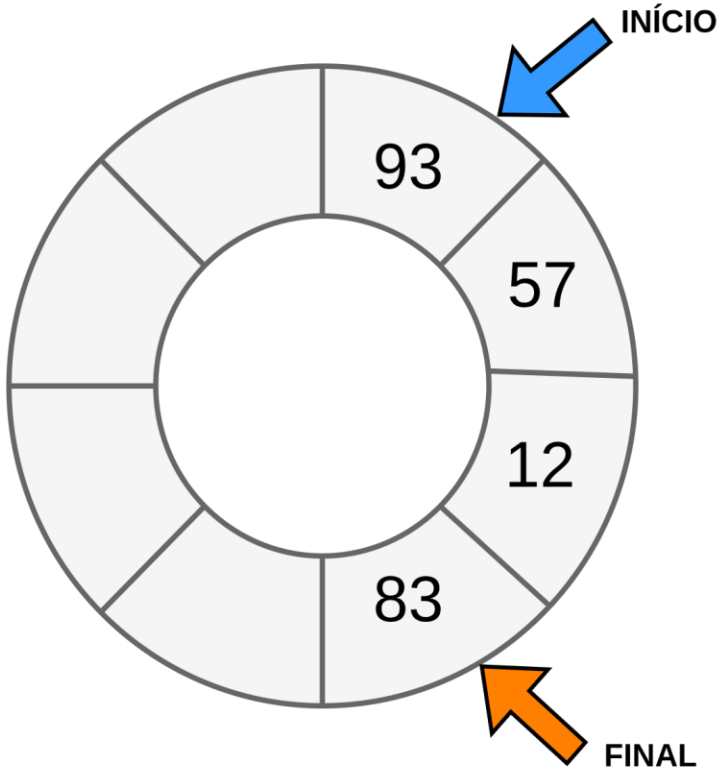
Filas Estáticas

- Na implementação circular, inserções podem continuar sendo realizadas normalmente, apenas movendo o indicador de fim para as posições vagas no início do vetor:



Filas Estáticas

- Podemos visualizar o mesmo vetor em um formato circular para abstrair melhor a ideia, como representado na imagem abaixo:



Filas Estáticas

- Na prática, tanto a definição de uma fila estática circular quanto a sua inicialização é idêntica à de fila estática tradicional.
- Então, teremos o **vetor propriamente dito**, **as variáveis para armazenar o índice do início e do final,** e, opcionalmente, **uma variável para monitorar o tamanho da fila**.

```c
#define TAMANHO_FILA 10

typedef struct estrutura_fila {
    int itens[TAMANHO_FILA];
    int inicio, final;
    int tamanho;
} Fila;


void inicializar_fila(Fila *fila) {
    fila->inicio = fila->final = -1;
    fila->tamanho = 0;
}
```

Filas Estáticas

- Na prática, tanto a definição de uma fila estática circular quanto a sua inicialização é idêntica à de fila estática tradicional.
- Então teremos o **vetor propriamente dito**, **as variáveis para armazenar o índice do início e do final**, e, opcionalmente, **uma variável para monitorar o tamanho da fila**.

```c
#define QUEUE_SIZE 10

typedef struct queue {
    int items[QUEUE_SIZE];
    int front, back;
    int size;
} Queue;


void initialize(Queue *queue) {
    queue->front = queue->back = -1;
    queue->size = 0;
}
```

Filas Estáticas

- A verificação de fila cheia na implementação circular é um pouco diferente, pois temos que verificar se o final da fila está imediatamente antes do início da fila. Porém, considerando a circularidade, essa condição pode ocorrer de várias formas.

```
int esta_cheia(Fila fila) {
    return (fila.final + 1) % TAMANHO_FILA == fila.inicio;
}
```

Filas Estáticas

- Mostrar o tamanho da fila é uma tarefa bem simples, basta apenas retornar o valor do campo tamanho da struct.

```
int tamanho_fila(Fila fila) {
    return fila.tamanho;
}
```

Filas Estáticas

- A implementação da função enfileirar nas filas estáticas circulares é um pouco diferente, pois **não precisamos mais fazer os deslocamentos**.
- Neste caso, **precisamos considerar a circularidade** sempre que formos enfileirar um elemento e o final já estiver na última posição válida do vetor.

```
int enfileirar(Fila *fila, int item) {
    if (esta_cheia(*fila)) {
        return 0;
    }

    if (fila->inicio == -1) {
        fila->inicio = 0;
    }

    fila->final = (fila->final + 1) % TAMANHO_FILA;

    fila->itens[fila->final] = item;
    fila->tamanho++;

    return 1;
}
```

Filas Estáticas

- A implementação da função desenfileirar precisa considerar dois casos principais:
  - A fila contém apenas **um elemento**.
  - A fila contém **mais de um elemento**.
- Se contiver apenas um, **basta removê-lo** e **inicializar a fila novamente**.
- Senão, **precisamos verificar se o início estava na última posição válida do vetor**, pois é preciso considerar a circularidade.

```c
int desenfileirar(Fila *fila, int *item) {
    if (esta_vazia(*fila)) {
        return 0;
    }

    *item = fila->itens[fila->inicio];

    if (fila->inicio == fila->final) {
        inicializar_fila(fila);
    } else {
        fila->inicio = (fila->inicio + 1) % TAMANHO_FILA;
        fila->tamanho--;
    }

    return 1;
}
```

Filas Estáticas

- Para mostrar a fila toda, partimos do índice do início da fila e seguimos até encontrar o índice do final da fila, sempre considerando a circularidade.

```c
void mostrar_fila(Fila fila) {
    if (esta_vazia(fila)) {
        printf("A fila está vazia\n");
        return;
    }

    printf("Itens da fila: ");

    int i = fila.inicio;

    while (i != fila.final) {
        printf("%d ", fila.itens[i]);
        i = (i + 1) % TAMANHO_FILA;
    }

    printf("%d\n", fila.itens[fila.final]);
}
```

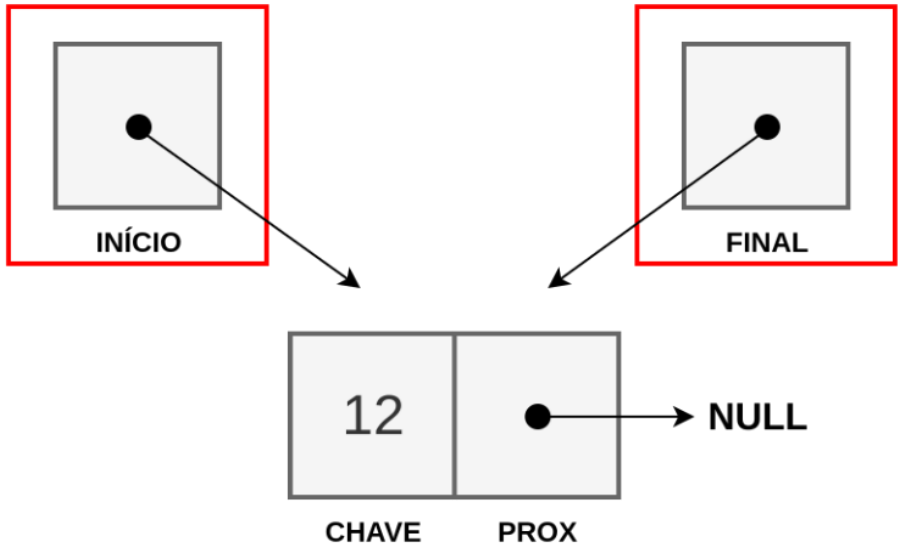
Filas Dinâmicas

Filas Dinâmicas

- Ao implementar uma fila utilizando lista encadeada como base, deve-se ter em mente que as inserções serão realizadas no final da estrutura:
  - Uma **lista simplesmente encadeada com ponteiro apenas para a cabeça** não é muito vantajosa, pois a inserção no final exigiria **percorrer toda a lista** para acessar o último nó.
- Então, para termos uma implementação eficiente, é necessário que possamos acessar diretamente os dois extremos da lista: **cabeça (início)** e **cauda (final)**.

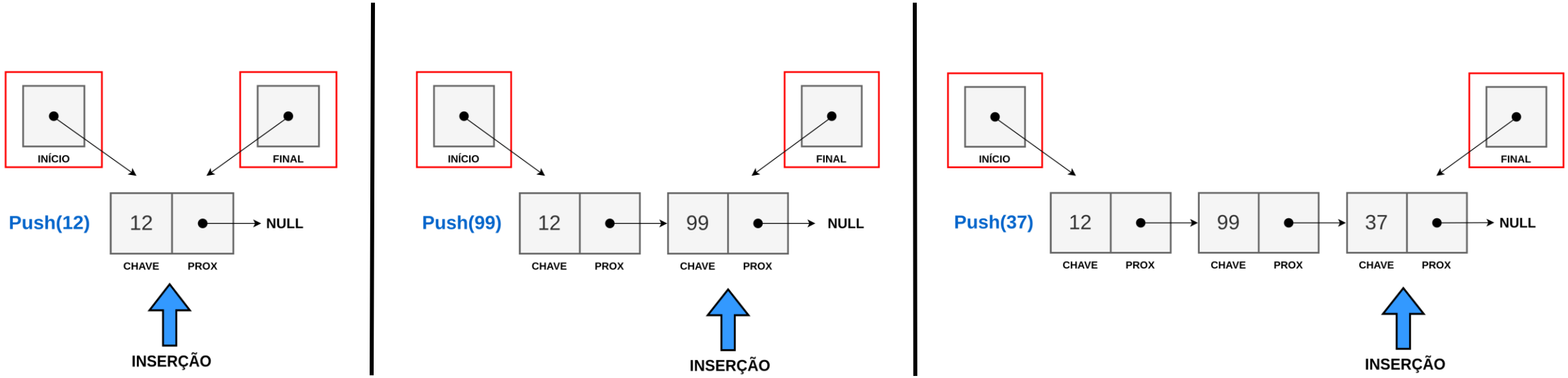
Filas Dinâmicas

- Inicialmente, os ponteiros de início e final da fila apontam para o único nó presente:



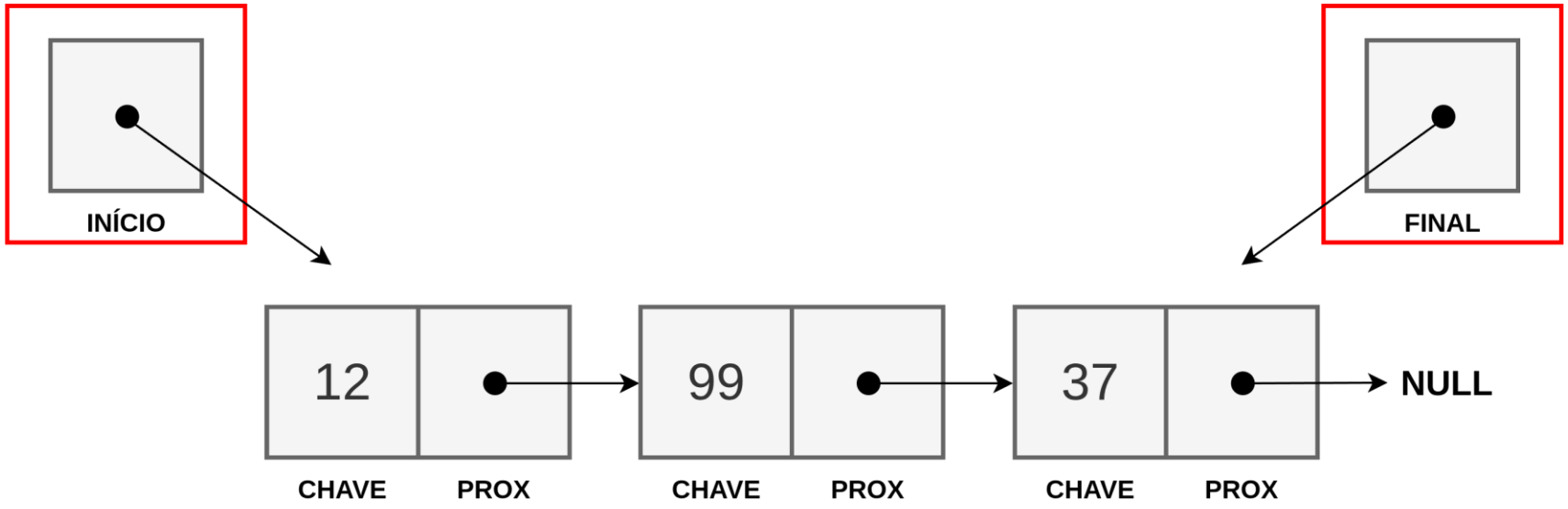
Filas Dinâmicas

- Com a inserção de novos elementos, novos nós são criados e ligados ao final da fila, com o devido ajuste do ponteiro que indica o final da fila:



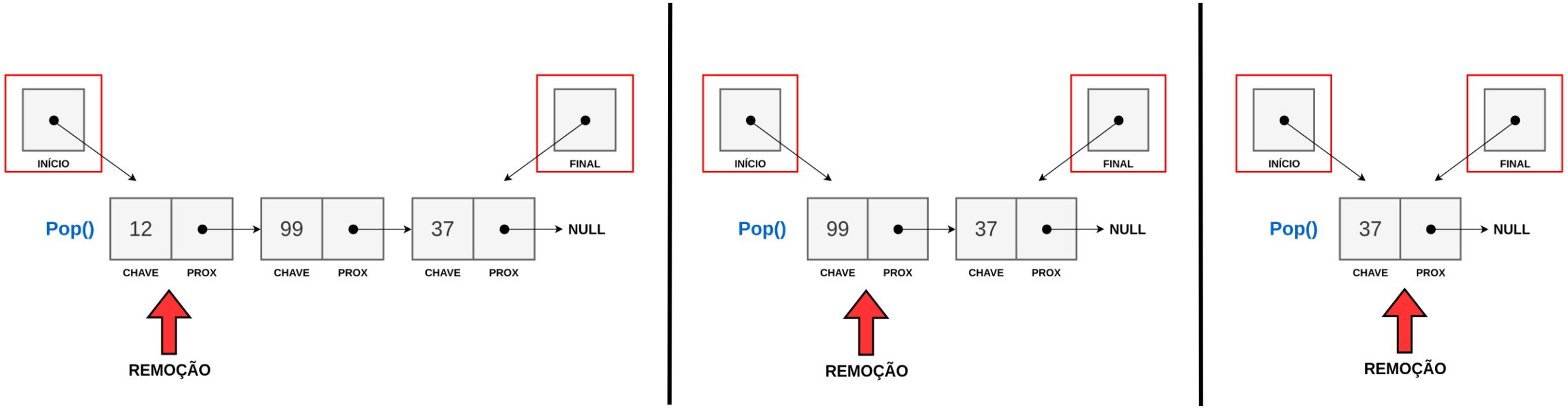
Filas Dinâmicas

- Após algumas inserções, a fila dinâmica atinge a situação mostrada na imagem abaixo, na qual o ponteiro de início permanece no primeiro nó, e o de final aponta para o último nó:



Filas Dinâmicas

- Então, ao remover elementos da fila, o nó que ocupa o início é simplesmente destruído, e o ponteiro de início passa a apontar para o próximo nó da fila:



Filas Dinâmicas

- Como vimos, a implementação dinâmica de fila emprega ponteiros para o **início** e **final**, os quais são agrupados em uma segunda struct.
- Além disso, podemos usar uma variável inteira para monitorar o tamanho da fila.
- Inicializar a fila envolve **apontar os ponteiros para NULL e atribuir 0 ao tamanho**.

```c
typedef struct no {
    char item;
    struct no *prox;
} *No;

typedef struct fila {
    No inicio, final;
    int tamanho;
} Fila;


void inicializar_fila(Fila *fila) {
    fila->inicio = fila->final = NULL;
    fila->tamanho = 0;
}
```

Filas Dinâmicas

- Como vimos, a implementação dinâmica de fila emprega ponteiros para o **início** e **final**, os quais são agrupados em uma segunda struct.
- Além disso, podemos usar uma variável inteira para monitorar o tamanho da fila.
- Inicializar a fila envolve **apontar os ponteiros para NULL e atribuir 0 ao tamanho**.

```
typedef struct node {
    char item;
    struct node *next;
} *Node;

typedef struct queue {
    Node front, back;
    int size;
} Queue;


void initialize_queue(Queue *queue) {
    queue->front = queue->back = NULL;
    queue->size = 0;
}
```

Filas Dinâmicas

- As operações disponíveis são exatamente as mesmas de filas estáticas, exceto pela função que verifica se a fila está cheia, pois aqui trabalhamos com alocação dinâmica.

```
int esta_vazia(Fila fila);
int inserir(Fila *fila, char item);
int remover(Fila *fila, char *item);
int espiar(Fila fila, char *item_inicio);
void mostrar_fila(Fila fila);
```

Filas Dinâmicas

- As operações disponíveis são exatamente as mesmas de filas estáticas, exceto pela função que verifica se a fila está cheia, pois aqui trabalhamos com alocação dinâmica.

```
int is_empty(Queue queue);
int push(Queue *queue, char item);
int pop(Queue *queue, char *item);
int peek(Queue queue, char *front_item);
void show_queue(Queue queue);
```

Filas Dinâmicas

- Para verificar se uma fila dinâmica está vazia, basta checar se o **ponteiro de início da fila é NULL**, isto é, não aponta para nenhum nó.

```
int esta_vazia(Fila fila) {
    return fila.inicio == NULL;
}
```

Filas Dinâmicas

- Para inserir um elemento na fila, cria-se um novo **nó**. Em seguida, verifica-se se o ponteiro de início é **NULL**. Se for, então o novo nó passa a ser o **início**.
- Caso contrário, o novo nó é ligado como **próximo nó do elemento no final da fila**.
- Por fim, o **ponteiro de final** é atualizado, passando a apontar para o novo nó.

```c
int inserir(Fila *fila, char item) {
    No no = (No) malloc(sizeof(struct no));

    if (no == NULL) {
        return 0;
    }

    no->item = item;
    no->prox = NULL;

    if (fila->inicio == NULL) {
        fila->inicio = no;
    } else {
        fila->final->prox = no;
    }

    fila->final = no;
    fila->tamanho++;

    return 1;
}
```

Filas Dinâmicas

- Para remover um elemento, primeiro salva-se o **valor armazenado no nó do início da fila**, bem como o **ponteiro para esse nó**.
- Então, o ponteiro de **início** é atualizado para apontar para o **próximo nó da fila**.
- Por fim, o nó que estava no início é **liberado**. Caso o ponteiro de início passe a ser **NULL**, o **final também deve ser**.

```
int remover(Fila *fila, char *item) {
    if (esta_vazia(*fila)) {
        return 0;
    }

    *item = fila->inicio->item;

    No p = fila->inicio;
    fila->inicio = p->prox;
    free(p);

    if (fila->inicio == NULL) {
        fila->final = NULL;
    }

    fila->tamanho--;

    return 1;
}
```

Filas Dinâmicas

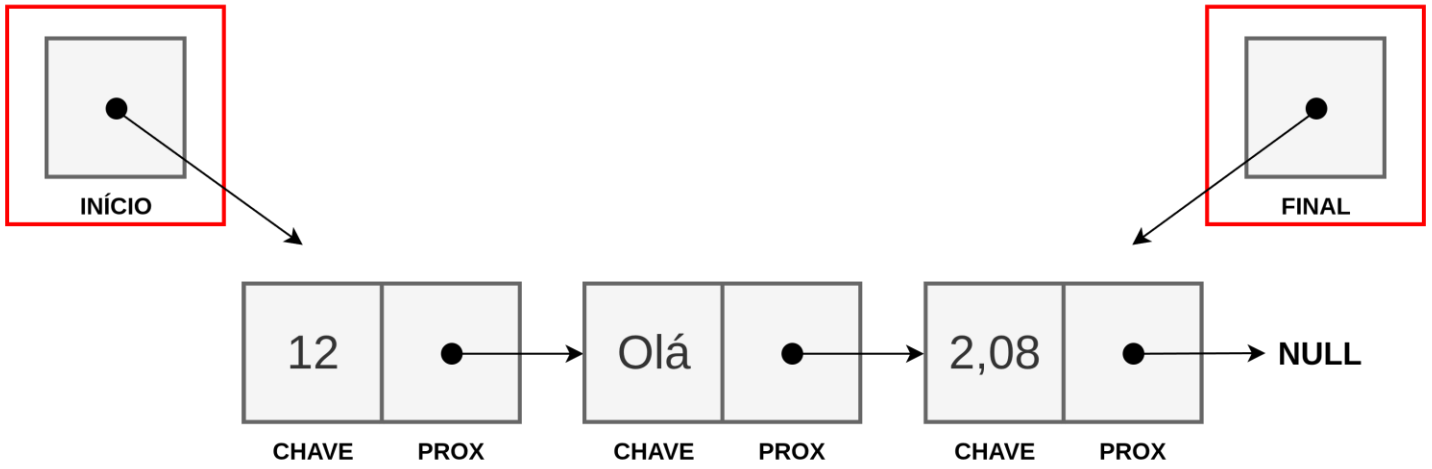
- Por último, é possível definir uma função auxiliar para **exibir os elementos da fila**.
- No entanto, é importante observar que essa implementação **fere o comportamento de uma fila**, pois o correto seria **remover os elementos para acessá-los**, respeitando o princípio **FIFO**.

```
void mostrar_fila(Fila fila) {
    if (esta_vazia(fila)) {
        printf("A fila está vazia\n");
        return;
    }

    printf("A fila tem %d itens: ", fila.tamanho);

    No p = fila.inicio;

    while(p != NULL) {
        printf("%c ", p->item);
        p = p->prox;
    }

    printf("\n");
}
```

Filas Heterogêneas

Filas Heterogêneas

- Até então, vimos estruturas homogêneas, o que é mais intuitivo em **C**. Porém, também é possível definir **estruturas heterogêneas**, isto é, que aceita **diferentes tipos de dados**.
- Uma forma simples de atingir esse objetivo é utilizar o tipo especial **union**;



Filas Heterogêneas

- O tipo **union** é de uma estrutura especial que permite armazenar diferentes dados no mesmo local de memória.
- Diferente da **struct**, onde cada membro tem seu próprio espaço, na **union**, todos os membros compartilham o mesmo endereço, e o tamanho total é o do maior campo.

```
#define TIPO_INT 1
#define TIPO_CHAR 2
#define TIPO_FLOAT 3

typedef union {
    int item_int;
    char item_char;
    float item_float;
} Item;

typedef struct node {
    int tipo_item;
    Item item;
    struct node *prox;
} *No;

typedef struct queue {
    No inicio, final;
    int tamanho;
} Fila;
```

Filas Heterogêneas

- A implementação de uma fila heterogênea mantém-se praticamente a mesma, exceto pelo fato de que se permite armazenar diferentes tipos de dado.
- Como o tipo de dado a ser armazenado pode variar, neste caso, entre **int**, **char** e **float**, precisamos de um campo para monitorar qual o tipo foi armazenado. Aqui consideramos o campo **tipo_item**.

```c
int enfileirar(Fila *fila, int tipo_item, Item item) {
    No no = (No) malloc(sizeof(struct node));

    if (no == NULL) {
        return 0;
    }

    no->tipo_item = tipo_item;
    no->item = item;
    no->prox = NULL;

    if (fila->inicio == NULL) {
        fila->inicio = no;
    } else {
        fila->final->prox = no;
    }

    fila->final = no;
    fila->tamanho++;

    return 1;
}
```

Filas Heterogêneas

- Note também que estamos passando como argumento da função o item a ser armazenado como sendo do tipo union.

```
int enfileirar(Fila *fila, int tipo_item, Item item) {
    No no = (No) malloc(sizeof(struct node));

    if (no == NULL) {
        return 0;
    }

    no->tipo_item = tipo_item;
    no->item = item;
    no->prox = NULL;

    if (fila->inicio == NULL) {
        fila->inicio = no;
    } else {
        fila->final->prox = no;
    }

    fila->final = no;
    fila->tamanho++;

    return 1;
}
```

Filas Heterogêneas

- Também precisamos nos atentar ao mostrar um item, pois **precisamos saber qual o tipo de dado armazenado para usarmos a função *printf()* da forma correta.**

```
void mostrar_item(int tipo_item, Item item) {
    switch (tipo_item) {
        case TIPO_INT:
            printf("%d ", item.item_int);
            break;
        case TIPO_CHAR:
            printf("%c ", item.item_char);
            break;
        case TIPO_FLOAT:
            printf("%.2f ", item.item_float);
            break;
        default:
            printf("Tipo inválido!\n");
    }
}
```

Filas Heterogêneas

- Uma desvantagem de utilizar **union** é que o espaço de memória alocado corresponde sempre ao maior tipo possível, mesmo quando não é utilizado.
- Para resolver esse problema, também é possível utilizar o **tipo void**, isto é, um ponteiro genérico que **pode apontar para qualquer tipo de dado.**

```
#define TIPO_INT 1
#define TIPO_CHAR 2
#define TIPO_FLOAT 3

typedef struct no {
    int tipo_item;
    void *item;
    struct no *prox;
} *No;

typedef struct fila {
    No inicio, final;
    int tamanho;
} Fila;
```

Filas Heterogêneas

- A implementação de uma fila heterogênea utilizando o **tipo void** difere da que utilizamos o **tipo union** apenas nos argumentos que enviamos para função.
- Neste caso, enviamos **o ponteiro void**.

```c
int enfileirar(Fila *fila, int tipo_item, void *item) {
    No no = (No) malloc(sizeof(struct no));

    if (no == NULL) {
        return 0;
    }

    no->tipo_item = tipo_item;
    no->item = item;
    no->prox = NULL;

    if (fila->inicio == NULL) {
        fila->inicio = no;
    } else {
        fila->final->prox = no;
    }

    fila->final = no;
    fila->tamanho++;

    return 1;
}
```

Filas Heterogêneas

- Mostrar um item com o tipo void não é diferente, ainda temos que nos atentar, pois **precisamos saber qual o tipo de dado armazenado para usarmos a função *printf()* da forma correta.**

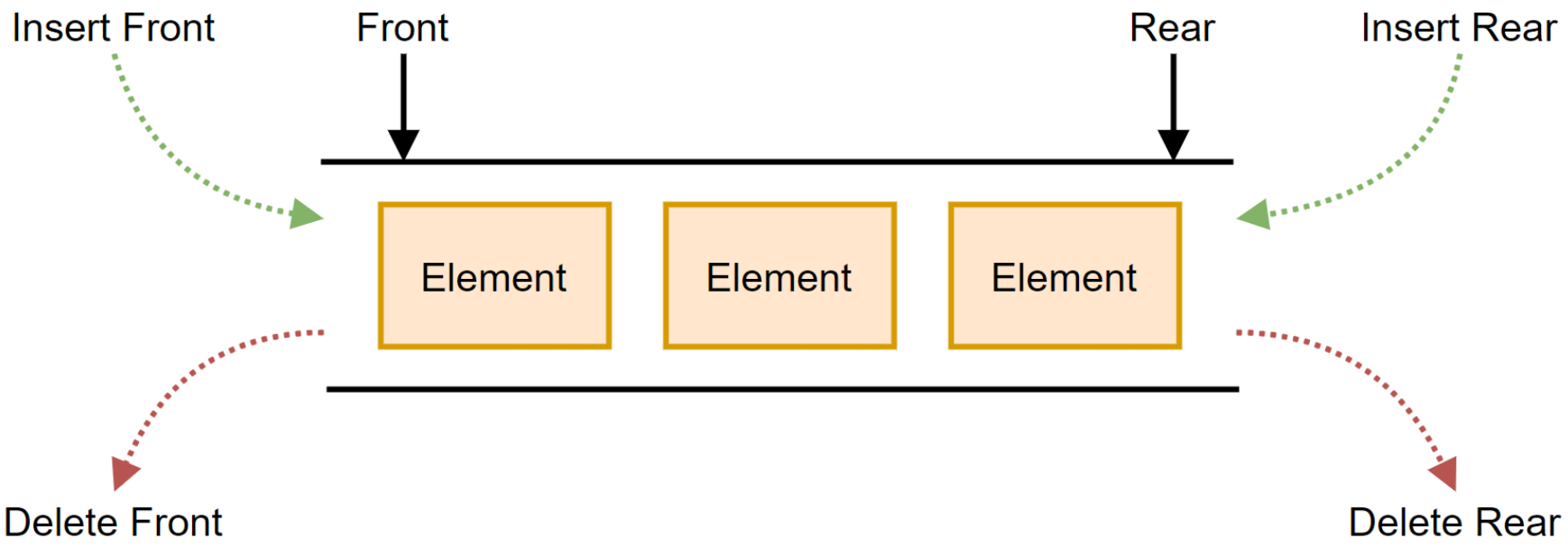
```
void mostrar_item(int tipo_item, void *item) {
    switch (tipo_item) {
        case TIPO_INT:
            printf("%d ", *(int *) item);
            break;
        case TIPO_CHAR:
            printf("%c ", *(char *) item);
            break;
        case TIPO_FLOAT:
            printf("%.2f ", *(float *) item);
            break;
        default:
            printf("Tipo inválido!\n");
    }
}
```

Variações de Fila

Deque

- A fila de duas pontas, ou **Double-Ended Queue (Deque)**, é uma estrutura que permite a inserção e remoção de elementos em **ambas as extremidades**.
- Dessa forma, a estrutura **deque** pode ser tratada como duas filas, permitindo inverter o sentido de uso da estrutura conforme a necessidade.
- De certa forma, é uma estrutura que **generaliza os conceitos de pilhas e filas**.

Deque


Insert Front
Front
Rear
Insert Rear
Element
Element
Element
Delete Front
Delete Rear

Deque

- Vimos que para implementar uma fila dinâmica tradicional precisamos, no mínimo, que ela seja simplesmente encadeada e que contenha ponteiros para início e final.
- No entanto, como uma **deque** pode inserir e remover os elementos em ambas as extremidades, é vantajoso que ela seja duplamente encadeada.

```c
typedef struct no {
    int item;
    struct no *anterior, *proximo;
} *No;

typedef struct deque {
    No inicio, final;
    int tamanho;
} Deque;


void inicializar(Deque *deque) {
    deque->inicio = deque->final = NULL;
    deque->tamanho = 0;
}
```

Deque

- Vimos que para implementar uma fila dinâmica tradicional precisamos, no mínimo, que ela seja simplesmente encadeada e que contenha ponteiros para início e final.
- No entanto, como uma **deque** pode inserir e remover os elementos em ambas as extremidades, é vantajoso que ela seja duplamente encadeada.

```
typedef struct node {
    int item;
    struct node *prev, *next;
} *Node;

typedef struct deque {
    Node front, back;
    int size;
} Deque;


void initialize(Deque *deque) {
    deque->front = deque->back = NULL;
    deque->size = 0;
}
```

Deque

- Teremos as mesmas operações da fila tradicional, acrescidas das funções para **inserir no início**, **remover no final** e **espiar o último elemento**.

```
int esta_vazio(Deque deque);
int inserir_inicio(Deque *deque, int item);
int inserir_final(Deque *deque, int item);
int remover_inicio(Deque *deque, int *item);
int remover_final(Deque *deque, int *item);
int ver_inicio(Deque deque, int *item_inicio);
int ver_final(Deque deque, int *item_final);
void mostrar_deque(Deque deque);
```

Deque

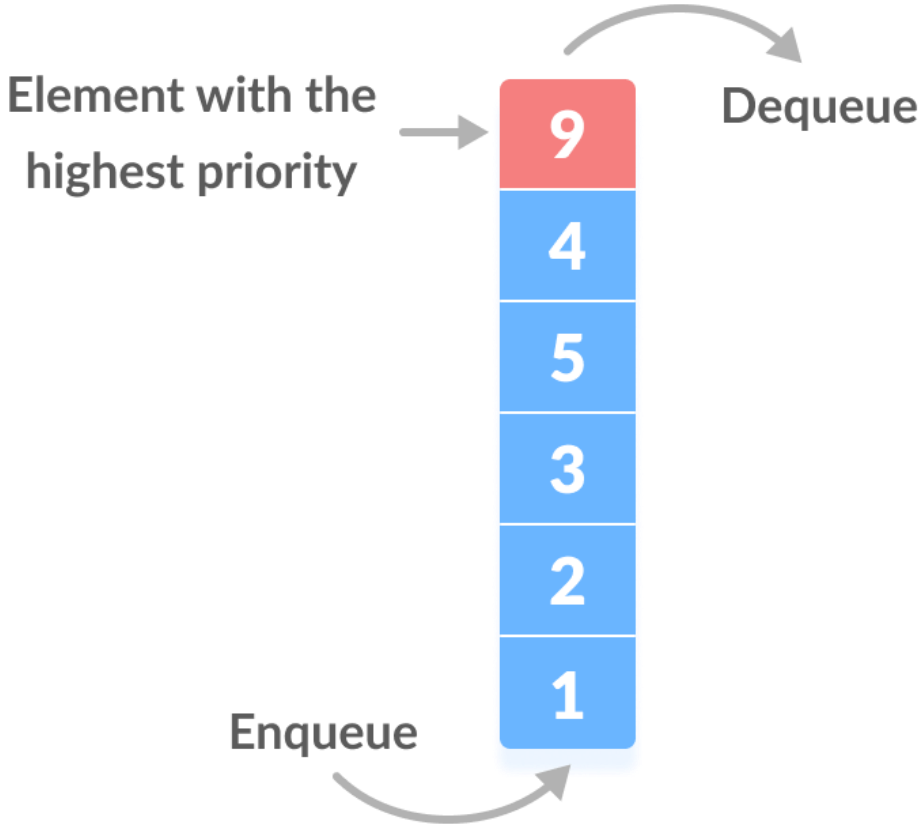
- Teremos as mesmas operações da fila tradicional, acrescidas das funções para **inserir no início (push_front)**, **remover no final (pop_back)** e **espiar o último elemento (peek_back)**.

```
int is_empty(Deque deque);
int push_front(Deque *deque, int item);
int push_back(Deque *deque, int item);
int pop_front(Deque *deque, int *item);
int pop_back(Deque *deque, int *item);
int peek_front(Deque deque, int *front_item);
int peek_back(Deque deque, int *back_item);
void show_deque(Deque deque);
```

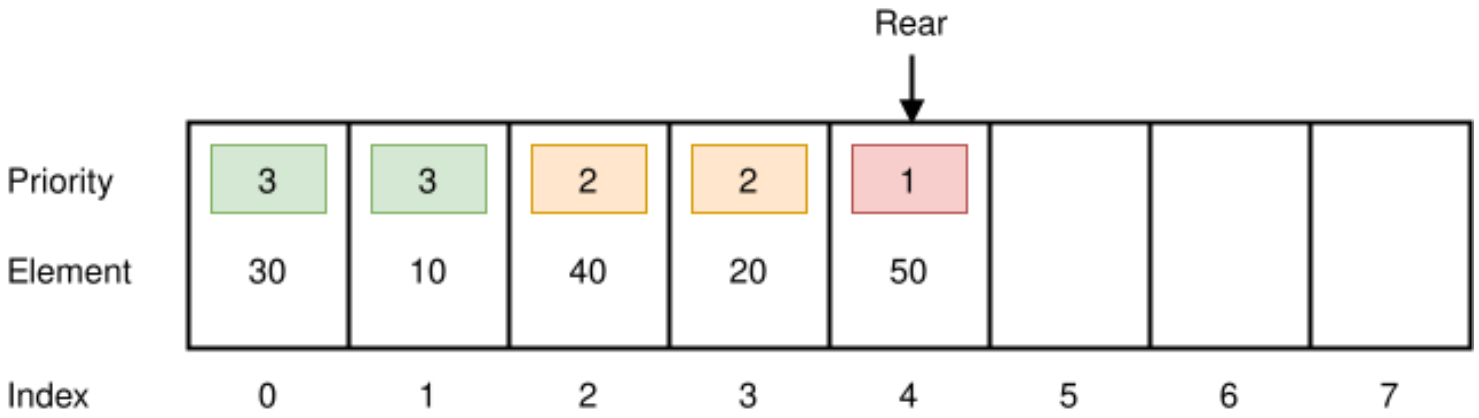
Fila de Prioridade

- Uma fila de prioridades é um tipo especial de fila em que os elementos são **organizados de acordo com sua prioridade**. Assim, o elemento de **maior prioridade é removido primeiro**.
  - Dependendo da implementação, valores menores podem indicar maior prioridade, ou o contrário.
- As remoções continuam sendo realizadas no início da fila, como em uma fila comum, porém as inserções consideram a prioridade dos elementos.
- Filas de prioridade podem ser implementadas com as abordagens já vistas, mas de forma mais eficiente utilizando **heap**, estrutura que veremos mais adiante na disciplina.

Fila de Prioridade



Fila de Prioridade



Fila de Prioridade

- A implementação envolve, essencialmente, o(s) item(s) que você quer armazenar e um indicador de prioridade.
- Neste caso, utilizamos uma variável inteira chamada **prioridade**. Consideramos que:
  - **1:** Indica **alta prioridade**;
  - **2:** Indica **média prioridade**; e
  - **3:** Indica **baixa prioridade**.

```c
#define TAM_MAX 30

#define ALTA_PRIORIDADE 1
#define MEDIA_PRIORIDADE 2
#define BAIXA_PRIORIDADE 3

typedef struct no {
    int prioridade;
    char item[TAM_MAX];
    struct no *prox;
} *No;

typedef struct fila {
    No inicio;
    int tamanho;
} FilaPrioridade;
```

Fila de Prioridade

- A implementação envolve, essencialmente, o(s) item(s) que você quer armazenar e um indicador de prioridade.
- Neste caso, utilizamos uma variável inteira chamada **prioridade**. Consideramos que:
  - **1:** Indica **alta prioridade**;
  - **2:** Indica **média prioridade**; e
  - **3:** Indica **baixa prioridade**.

```c
#define MAX_LEN 30

#define HIGH_PRIORITY 1
#define MEDIUM_PRIORITY 2
#define LOW_PRIORITY 3

typedef struct node {
    int priority;
    char item[MAX_LEN];
    struct node *next;
} *Node;

typedef struct queue {
    Node front;
    int size;
} PriorityQueue;
```

Fila de Prioridade

- A inserção de elementos deve considerar, portanto, o nível de prioridade do item armazenado. Por exemplo, se o item tem:
  - **Prioridade Alta:** Deverá ser inserido antes dos itens de prioridade média e prioridade baixa.
  - **Prioridade Média:** Deverá ser inserido depois dos itens de prioridade alta e antes dos itens de prioridade baixa.
  - **Prioridade Baixa:** Deverá ser inserido após os itens de prioridade média e prioridade alta.

```c
#define TAM_MAX 30

#define ALTA_PRIORIDADE 1
#define MEDIA_PRIORIDADE 2
#define BAIXA_PRIORIDADE 3

typedef struct no {
    int prioridade;
    char item[TAM_MAX];
    struct no *prox;
} *No;

typedef struct fila {
    No inicio;
    int tamanho;
} FilaPrioridade;
```

Aplicações

Aplicações

- As **filas** podem ser utilizadas para resolver diversos problemas.
- Um uso comum das filas é **simular filas reais**, podendo ser utilizadas para criar aplicações de atendimento, como em hospitais, ou para organizar **filas de tarefas em uma API**.
- Muitos algoritmos utilizam filas como base para sua implementação, como é o caso da **ordenação por distribuição**, também conhecida como Radix Sort.
- Além disso, as filas são utilizadas pelos sistemas operacionais em tarefas simples, como em uma **fila de impressão**, mas também complexas, como o **escalonamento de processos**.

Ordenação por Distribuição

- Considerando valores na base **B = 10**, ou seja, números decimais, o algoritmo Radix Sort ordena uma lista utilizando **10 filas**, cada uma representando um dígito de **0 a 9**.
- Para determinar quantas iterações serão necessárias, basta identificar **qual número da lista possui a maior quantidade de dígitos**.
- Em cada iteração, os números são distribuídos nas filas de acordo com um dígito específico, começando pelas **unidades**, depois **dezenas**, **centenas** etc.
  - Cada número é colocado na fila correspondente ao valor do dígito analisado.

Ordenação por Distribuição

- Após todos os números serem distribuídos, a lista é **reconstruída juntando os elementos das filas na ordem de 0 a 9**.
- Esse processo de distribuição e reconstrução é repetido para cada posição de dígito.
- Ao final da última iteração, quando todos os dígitos tiverem sido considerados, **a lista estará completamente ordenada**.

Ordenação por Distribuição

LISTA: 19 13 05 27 01 26 31 16 02 09 11 21 60 07

1ª iteração **(unidades)**

Filas	Valores
F0	60
F1	01, 31, 11, 21
F2	02
F3	13
F4	
F5	05
F6	26, 16
F7	27, 07
F8	
F9	19, 09

LISTA ATUALIZADA:

60 01 31 11 21 02 13 05 26 16 27 07 19 09

Ordenação por Distribuição

LISTA: 60 01 31 11 21 02 13 05 26 16 27 07 19 09

2ª iteração **(dezenas)**

Filas	Valores
F0	01, 02, 05, 07, 09
F1	11, 13, 16, 19
F2	21, 26, 27
F3	31
F4	
F5	
F6	60
F7	
F8	
F9	

LISTA ATUALIZADA:

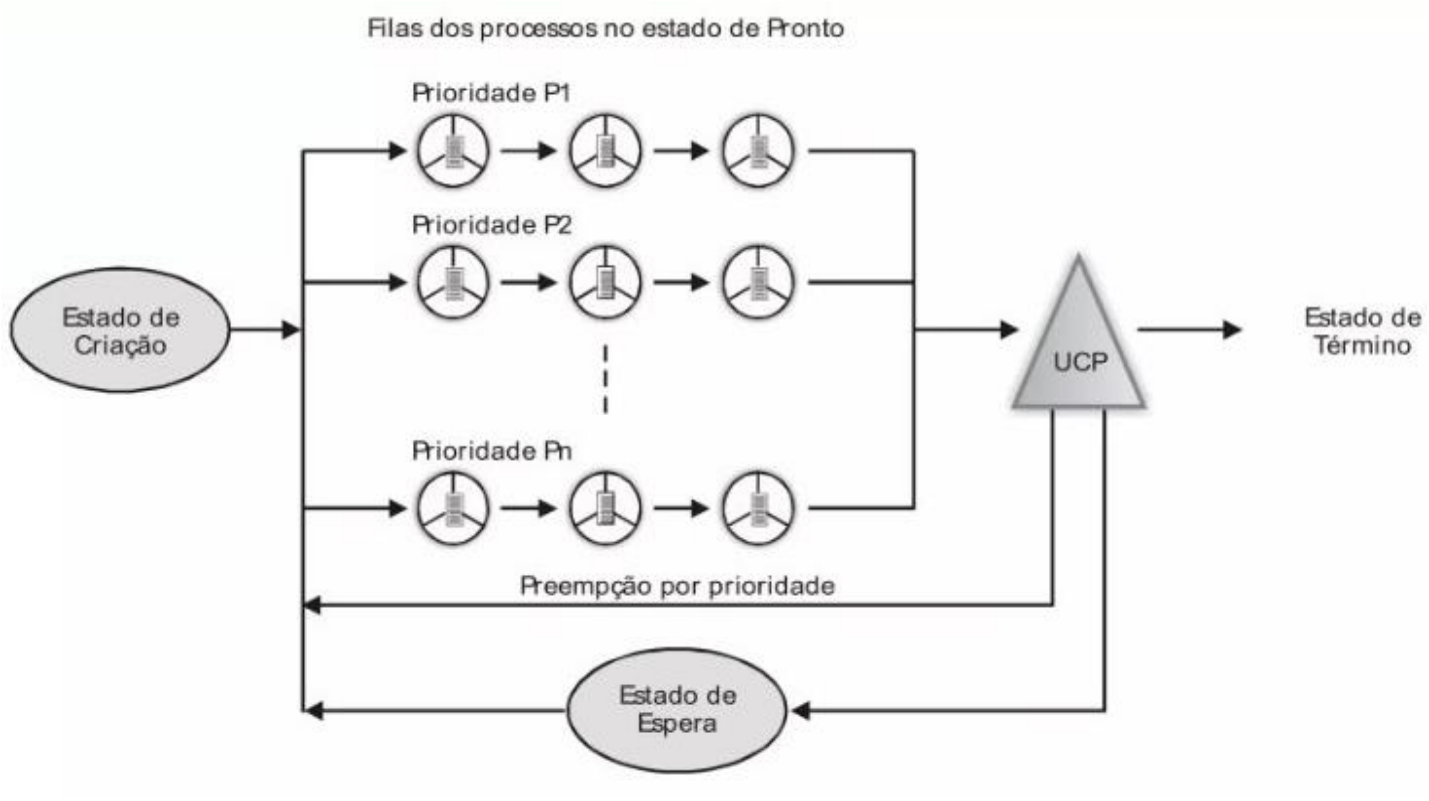
01 02 05 07 09 11 13 16 19 21 26 27 31 60

Escalonamento de Processos

- O sistema operacional é responsável por realizar o escalonamento de processos, que consiste em decidir qual processo da fila de prontos irá utilizar a CPU e por quanto tempo.
- Quando vários processos precisam ser executados, eles são colocados em uma fila de prontos, aguardando sua vez de serem processados.
- O escalonador do sistema operacional analisa essa fila e seleciona o próximo processo a ser executado, podendo levar em consideração fatores como:
  - Ordem de chegada, prioridade e tempo de execução.

Escalonamento de Processos

- Nesse contexto, as filas desempenham um papel fundamental:



Ficou alguma dúvida?

E-mail para contato: giovani.candido@unesp.br e lf.sementille@unesp.br

Materiais e Avisos: Google Classroom